Universidade Federal do Rio Grande do Norte
Instituto Metrópole Digital - IMD
Tecnologia da Informação

Prof. Mateus Alves de Oliveira
mateus.oliveira@imd.ufrn.br

LINGUAGEM DE PROGRAMAÇÃO II

Atividade Prática 05
Leitura e Escrita de Arquivos

CONTEXTO

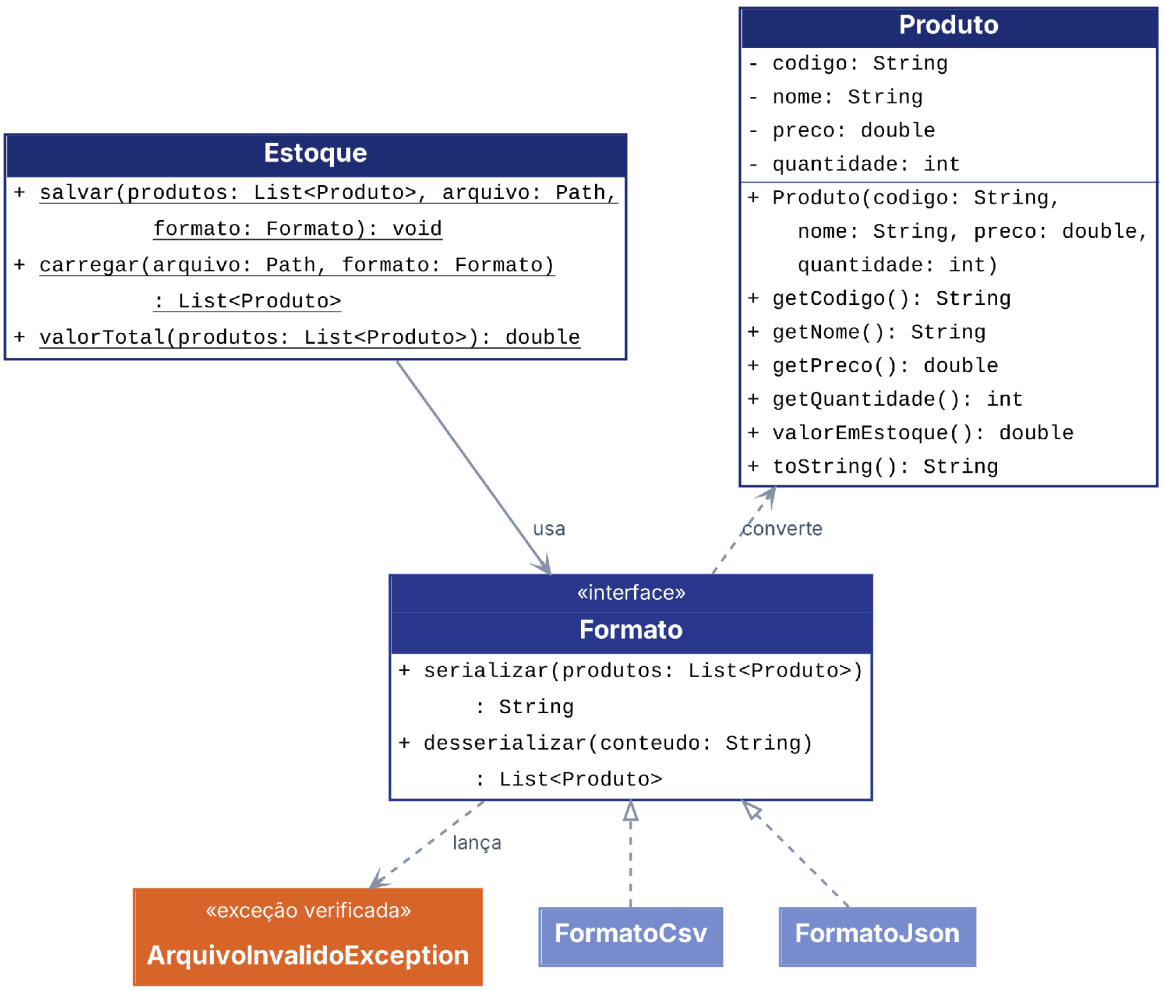
Na **Aula 11** de **Leitura e Escrita de Arquivos**, vimos como ler e gravar arquivos com as classes **`Path`** e **`Files`**, como tratar a **`IOException`** e como trabalhar com diferentes formatos, como **XML**, **CSV** e **JSON**.

Nesta prática, uma papelaria guarda o seu estoque no arquivo **`produtos.csv`** e quer exportá-lo para JSON. Você vai implementar as classes do diagrama abaixo. A interface **`Formato`** permite que a classe **`Estoque`** grave e leia o estoque em qualquer formato de arquivo, sem saber qual está usando.

Junto com este enunciado, você recebe dois arquivos:
`Formato.java`: a interface que **`FormatoCsv`** e **`FormatoJson`** devem implementar;
`produtos.csv`: o estoque inicial, com 5 produtos.

DIRETRIZES

- Use apenas o JDK (pacote **`java.nio.file`**), sem bibliotecas externas.
- Não altere a interface **`Formato.java`**.
- A classe **`Estoque`** deve conhecer apenas a interface **`Formato`**, nunca **`FormatoCsv`** ou **`FormatoJson`**.
- Coloque **`produtos.csv`** na raiz do projeto.



O QUE FAZER:

1. Implemente a classe **`Produto.java`** conforme o diagrama.

2. Crie a exceção **`ArquivoInvalidoException.java`**:

Ela deve ser uma exceção **verificada**. Sem ela, a interface **`Formato`** não compila.

3. Implemente a classe **`FormatoCsv.java`**:

O método **`serializar`** deve gerar um texto no mesmo formato de **`produtos.csv`**, com cabeçalho. O método **`desserializar()`** faz o caminho inverso. Uma linha com dado inválido (ex: preço **`abc`**) deve lançar **`ArquivoInvalidoException`**.

4. Implemente a classe **`Estoque.java`**:

Os métodos **`salvar()`** e **`carregar()`** gravam e leem o arquivo usando o **`Formato`** recebido como parâmetro. O método **`valorTotal()`** soma o valor em estoque de todos os produtos.

5. Crie a classe **`Main.java`**:

No **`main()`**, carregue **`produtos.csv`**, liste os produtos e mostre o valor total em estoque. Confira: o valor total é **R$ 4823.20**.

6. Implemente a classe **`FormatoJson.java`** e exporte o estoque:

No **`main()`**, use o **`Estoque`** para converter **`produtos.csv`** em **`produtos.json`**. Abra o arquivo gerado e confira as regras do JSON. Depois, carregue **`produtos.json`** e mostre o valor total de novo: ele deve ser o mesmo do item 5.

7. Crie a classe **`TesteEstoque.java`** com testes usando **`assert`**:

(a) Ida e volta: salvar uma lista e carregá-la de volta devolve a mesma lista, em CSV e em JSON;

(b) Carregar um arquivo inexistente lança **`NoSuchFileException`**;

(c) Um CSV com preço **`abc`** lança **`ArquivoInvalidoException`**;

(d) Um JSON incompleto lança **`ArquivoInvalidoException`**.

Execute com **`java -ea TesteEstoque`**. Todos os testes devem passar: se algum teste falhar, investigue a causa e corrija o código, não o teste.

ENTREGÁVEL

Um arquivo **`.zip`** contendo:

- as classes **`.java`** da prática (**`Produto`**, **`ArquivoInvalidoException`**, **`Formato`**, **`FormatoCsv`**, **`FormatoJson`**, **`Estoque`**, **`Main`** e **`TesteEstoque`**);
- o arquivo **`produtos.json`** gerado pelo seu programa;
- **`README.md`**, com:
- instruções de como compilar e executar o projeto e os testes (comandos **`javac`**, **`java`** e **`java -ea`**);
- a versão do JDK utilizada (verificar com **`java -version`**);
- a resposta à pergunta: **o que precisou mudar na classe Estoque para trocar CSV por JSON? Por quê?**

Critérios de avaliação (total: 10,0 pontos)

CRITÉRIO	PONTUAÇÃO
Classe **`Produto`** e exceção **`ArquivoInvalidoException`**.	1,0
`FormatoCsv`: grava e lê corretamente, rejeitando linhas inválidas.	2,0
`Estoque`: **`salvar`**, **`carregar`** e **`valorTotal`** usando apenas a interface **`Formato`**.	2,0
`FormatoJson`: gera um JSON válido e lê de volta.	2,0
`TesteEstoque`: testes com **`assert`** passando.	2,0
`README.md` completo.	1,0
Inclusão dos arquivos *`bytecode`* no ZIP entregável.	**- 2,0**